

Your Assignment — Northwind Support Desk

This is the only assignment document. Read it start to finish before you write any code. [README.md](#) covers how to run the project and how the codebase is laid out.

What we are asking you to do

You are joining a team that already has a working Support Ticket Management System (.NET 10 Web API + React 19 + SQL Server). You have **four tasks**, in this order:

#	Task	What "done" looks like	Rough effort
1	Fix a reported defect, then add server-side filtering	The bug reproduces for you, you fix the <i>cause</i> , and the fix has a test that fails before it and passes after; then every ticket-list filter is applied by the API, in the database, with a correct count	~2.5 h
2	Build the Ticket Escalation & Assignment feature	New tickets are triaged automatically (priority, SLA due date, assigned agent); tickets can be escalated; the UI shows all of it	~7 h
3	Add login and authentication	Agents log in with email + password; the API rejects unauthenticated calls; the UI has a login screen and a logout	~4 h
4	Dockerise the API	<code>docker compose up</code> on a clean checkout starts SQL Server and the API, migrated and seeded, with no secret baked into the image	~1.5 h

Time budget: two working days, about 13–17 hours. Do not go beyond that. We would much rather see three tasks done well and the last honestly marked "not finished" than four rushed ones. If you run out of time, the priority order is **Task 1** → **Task 2** → **Task 3** → **Task 4**, and one piece of Task 2 (the SLA summary report) is explicitly optional — see [Task 2, step 5](#).

AI tools are fully allowed. ChatGPT, Claude, Copilot, Cursor — whatever you normally use. No penalty for using them, no bonus for avoiding them. The only rule: you must understand and be able to defend every line you submit. See [Using AI](#).

Contents

- [Step 0 — Get oriented \(about 1 hour\)](#)
- [Task 1 — Fix the defect and add server-side filtering](#)
- [Task 2 — Ticket Escalation & Assignment](#)
- [Task 3 — Login and authentication](#)
- [Task 4 — Dockerise the API](#)
- [Where to focus your effort](#)
- [Suggested two-day plan](#)
- [Definition of done](#)
- [What to write in your README](#)

- [Using AI](#)
- [How to submit](#)
- [Out of scope](#)

Step 0 — Get oriented (about 1 hour)

Do not skip this. An hour here saves three later.

1. **Get it running** — follow [README.md](#) → [Quick start](#): `npm install`, `npm run db:up`, then `npm run api` and `npm run web` in two terminals. The API, the web app and `npm test` should all work *before* you change anything.
2. **Use the app** like a support agent: list tickets, filter them, open one, create one, change a status, assign an agent, look at a customer.
3. **Call the API directly** at `https://localhost:7043/swagger` .
4. **Read the code**, in this order:
 - `apps/api/src/SupportDesk.Domain/Aggregates/Tickets/Ticket.cs` and the enums next to it
 - `apps/api/src/SupportDesk.Application/Features/Tickets/` — the existing use cases (`Commands/` and `Queries/`)
 - `apps/api/src/SupportDesk.Infrastructure/Queries/TicketQueries.cs` — how queries are built
 - `apps/api/src/SupportDesk.Infrastructure/Data/Configurations/` — how the database is mapped
 - `apps/web/src/features/tickets/` — pages, components, hooks
 - `apps/api/tests/SupportDesk.UnitTests/` — the test style we use, including the `FixedClock` and `TicketBuilder` test doubles you are expected to reuse
5. **Learn the seed data** (`apps/api/src/SupportDesk.Infrastructure/Data/SupportDbSeeder.cs` , or `db/seed.sql`). You will need it to check your own work:

Agent	Active	Open tickets / limit	Specializations
Alex Turner	yes	5 / 10	General, Technical
Ben Osei	yes	3 / 8	Technical, Outage
Priya Nair	yes	4 / 8	Outage, Security
Sara Lindqvist	yes	2 / 6	General, Billing
Marcus Doyle	yes	6 / 6 (full)	Billing, Security
Yuki Tanaka	no	0 / 8	Technical

Customers: Contoso, Adventure Works and Wide World Importers are **Premium**; Fabrikam, Northwind Traders and Tailspin Toys are Standard. Categories: General, Billing, Technical, Outage, Security.

Tip: in Development the API logs every SQL statement it runs. Keep that console visible while you click around — it is the fastest way to understand the app, and you will want it for Task 1.

Task 1 — Fix the defect and add server-side filtering

Our QA has filed a bug report against the ticket list, and the list's filters have never been implemented on the server. Fix the defect first, then build the filtering.

What we expect from you for the defect

1. **Reproduce it** yourself, so you know exactly what "broken" looks like.
2. **Find the root cause** — the line that is wrong, and *why* it produces this symptom.
3. **Fix the cause, not the symptom.** A fix that hides the effect (for example, filtering the data after loading it, forcing a page reload, or remounting a component) counts as *not fixed*, even if the screen looks right afterwards.
4. **Add a regression test** that fails against the original code and passes against yours.
5. **Write the root cause in one or two sentences** in the commit message or the PR description. "Fixed the list" tells us nothing. "The date was compared in local time instead of UTC, so tickets raised after 23:00 showed under the next day" tells us everything.

Commit the defect fix and the filtering **separately from each other and from the feature work**, so a reviewer can see each on its own.

1.1 DEFECT-117 — Filters sometimes do not take effect

Reported by: Support agent **Steps:** Open the ticket list with the browser's developer tools on the **Network** tab. Change the **Status** dropdown to `In progress`. Do not touch anything else. **Expected:** A new request to `GET /api/tickets` goes out, with `status=InProgress`. **Actual:** No request is sent. But if I then type something in the search box, a request suddenly goes out *with* the status I picked earlier. Clearing the search box makes it go stale again.

The API does not filter yet (see 1.2), so watch the requests rather than the rows: this defect is about what the web app sends.

How we will check your fix: changing any single filter sends a new request immediately; the app does not fire a request per keystroke while typing in search; the **new SLA filter you add in Task 2 behaves correctly too**, without you having to remember to wire it specially; your regression test asserts the request that gets sent.

1.2 Implement server-side filtering

The ticket list already has filter controls, and the web app sends their values to `GET /api/tickets`. The API binds them into `TicketQuery` — and then ignores them: today it only sorts and pages, so every filter returns the whole table. Build the filtering in the backend.

#	Requirement
FL-1	<code>status</code> , <code>priority</code> , <code>categoryId</code> , <code>customerId</code> and <code>assignedAgentId</code> are exact matches. Filtering by <code>High</code> returns High tickets only — not High <i>and above</i> .
FL-2	<code>unassignedOnly=true</code> returns only tickets with no assigned agent.
FL-3	<code>search</code> matches the ticket title, its reference (<code>TCK-0042</code>) or the customer's name. It is trimmed, and a blank search is ignored.
FL-4	Filters combine — every filter given must match (AND).

#	Requirement
FL-5	The database does the filtering. Filters are applied to the query <i>before</i> counting and paging, so <code>totalCount</code> , <code>totalPages</code> and the pager are right for the filtered set. No loading rows and filtering in C#. Read the SQL in the API log to check.
FL-6	It is the place the next filter goes. Task 2 adds <code>slaStatus</code> ; adding it should mean adding one more filter, not restructuring the query.

How we will check it: filtering by `High` returns only High tickets and the total count matches; filtering by `Low` does not return the whole table; status, category, customer, agent, unassigned-only and search each narrow the list, alone and combined; the pager is right; your tests cover the filtering path — at least the exact-match priority filter, a combination of filters, and a search that matches on customer name. The test project already references `Microsoft.EntityFrameworkCore.Sqlite` , if you want to run your queries against a real database.

Task 2 — Ticket Escalation & Assignment

The problem we are solving

Today a team lead reads every new ticket, decides how urgent it is, works out when it must be answered by, and picks someone to own it. That is slow and inconsistent. They want the system to make that decision, and they want to be able to **escalate** a ticket that is going badly, with a record of who escalated it and why.

2.1 The business rules

These are the complete rules. They are deliberately simple — **the challenge of this task is where you put them, not how clever they are.**

Rule	What it says
BR-1 · Priority	Use the priority the caller requested. If none was given, use <code>Medium</code> . But a ticket in a category flagged <code>ForcesCriticalPriority</code> (Security, Outage) is always <code>Critical</code> , whatever was requested.
BR-2 · SLA window	Due date = creation time + base window. <code>Critical</code> 4 h · <code>High</code> 8 h · <code>Medium</code> 24 h · <code>Low</code> 72 h.
BR-3 · Premium customers	A <code>Premium</code> customer's window is halved , but never shorter than 1 hour .
BR-4 · Specialists	A ticket in a category flagged <code>RequiresSpecialist</code> (Billing, Outage, Security) may only be assigned to an agent who has that category as a specialization.
BR-5 · Assignment	From the eligible agents — active, specialised where BR-4 applies, and strictly below their <code>MaxOpenTickets</code> — pick the one with the fewest open tickets (open = not <code>Resolved</code> and not <code>Closed</code>). Break ties deterministically (e.g. lowest id), so the same input always gives the same answer. If nobody is eligible, create the ticket unassigned with a reason. That is a normal outcome, not an error.

Rule	What it says
BR-6 • Rules live in data or configuration	The base windows, the premium multiplier, the 1-hour floor and the at-risk threshold must be changeable without hunting through code. apps/api/src/SupportDesk.Presentation/appsettings.Development.json already contains an (unused) Sla section, and the Categories table already has the RequiresSpecialist and ForcesCriticalPriority columns. Use them, or justify something better. if (category.Name == "Security") is not acceptable.
BR-7 • Escalation	Raises priority one level (Low → Medium → High → Critical). Recomputes the due date from the moment of escalation using BR-2 and BR-3. Re-evaluates the owner: keep the current agent if they are still eligible, otherwise reassign per BR-5. Requires a reason (5–500 characters). Records an immutable history row. A Critical ticket cannot be escalated further, and a Resolved or Closed ticket cannot be escalated at all.
BR-8 • SLA status	Always derived, never stored. No due date → NotApplicable . Resolved on or before the due date → Met . Resolved late, or unresolved and past due → Breached . Unresolved with 25% or less of the window remaining → AtRisk . Otherwise → WithinSla . "Now" must be injectable, so tests never depend on the wall clock.

2.2 Worked examples — check your implementation against these

Using the seeded data above. These are the answers we expect.

Input	Priority	Window	Due	Assigned to	Why
Adventure Works (Premium), Outage , requested Medium	Critical	2 h	now + 2 h	Ben Osei	Outage forces Critical; 4 h halved for Premium; Outage specialists are Ben (3 open) and Priya (4 open)
Contoso (Premium), Security , requested Low	Critical	2 h	now + 2 h	Priya Nair	Security forces Critical; Security specialists are Priya (4 open) and Marcus (full, 6/6 → not eligible)
Fabrikam (Standard), Billing , requested High	High	8 h	now + 8 h	Sara Lindqvist	Billing does not force Critical; Billing specialists are Sara (2 open) and Marcus (full)
Tailspin (Standard), Technical , no priority given	Medium	24 h	now + 24 h	Sara Lindqvist	Default Medium; Technical needs no specialist, so any active agent under their limit — Sara has the fewest open (2). Yuki is inactive and never eligible
Wide World (Premium), General , requested Low	Low	36 h	now + 36 h	Sara Lindqvist	72 h halved
A hypothetical 1 h base window for a Premium customer	—	1 h	now + 1 h	—	The floor in BR-3, not 30 minutes
Escalate a High ticket at 14:00 for a Standard customer	Critical	4 h	18:00	current agent if still eligible	Recomputed from the escalation time, not from creation

Input	Priority	Window	Due	Assigned to	Why
Escalate a <code>Critical</code> ticket	—	—	—	—	Rejected with <code>409</code>
Escalate a <code>Resolved</code> ticket	—	—	—	—	Rejected with <code>409</code>

2.3 What to build — backend

A. Triage on create. `POST /api/tickets` already exists. Extend it so creating a ticket applies BR-1 to BR-6 and returns the decision, including a short human-readable reason for each part of it:

```
{
  "id": 41, "reference": "TCK-0041", "priority": "Critical",
  "assignedAgent": { "id": 3, "fullName": "Priya Nair" },
  "dueAtUtc": "2026-09-27T11:00:00Z",
  "slaStatus": "WithinSla",
  "triage": {
    "appliedPriority": "Critical",
    "priorityReason": "Category 'Security' is always treated as Critical",
    "slaWindowHours": 2,
    "slaReason": "Critical base window 4h, halved for a Premium customer",
    "assignmentReason": "Fewest open tickets among Security specialists"
  }
}
```

When nobody could be assigned: `assignedAgent` is `null`, the call still returns `201`, and `assignmentReason` explains why — e.g. `"No available agent specialised in Security"`.

B. Escalation. `POST /api/tickets/{id}/escalate`

```
// Body before you do Task 3:
{ "reason": "Customer's whole site is down and the SLA is about to breach.",
  "escalatedBy": "alex.lead" }
```

Response	When
<code>200</code> + the updated ticket (same shape as above) + the escalation that was recorded	Success
<code>400</code>	Reason missing or outside 5–500 characters; <code>escalatedBy</code> missing
<code>404</code>	No such ticket
<code>409</code>	Already <code>Critical</code> , or <code>Resolved</code> / <code>Closed</code> — the message must say which

Once you have done Task 3, `escalatedBy` must come from the **authenticated user**, not from the request body. Say in your README which one you ended up with.

C. History. `GET /api/tickets/{id}/escalations` — newest first, showing the before/after priority, the before/after agent, the before/after due date, the reason, who, and when.

D. Persistence. A new table, created by an **EF Core migration you commit**:

```
TicketEscalations
  Id, TicketId (FK, indexed), FromPriority, ToPriority,
  FromAgentId, ToAgentId, FromDueAtUtc, ToDueAtUtc,
  Reason (500), EscalatedBy (100), EscalatedAtUtc
```

Name things your way if you prefer, but configure it properly (keys, lengths, required/optional, delete behaviour) and add whatever index your queries actually need — then say in your README which index you added and why.

E. SLA filtering. `GET /api/tickets` gains a `slaStatus` parameter (`?slaStatus=Breached`). Filtering, sorting, paging **and the count** must all happen in SQL. We will read the SQL your code generates — it is printed in the API console.

2.4 What to build — frontend

In `apps/web/src/features/tickets`:

1. **A reusable SLA indicator** showing `Within SLA` / `At risk` / `Breached` / `Met` / `No SLA`, used by **both** the list and the detail page. There is a list-only `SLAcell` today and it does not handle `At risk`. Do not convey the state by colour alone.
2. **An SLA status filter** in the filter bar, wired to the new API parameter.
3. **An escalation panel** on the ticket detail page: a reason field, validation matching the server's rules (so an invalid reason never reaches the API), a pending state while it saves, server errors shown to the user — **including the 409 cases** — and the ticket and its history refreshed on success without a full page reload.
4. **The escalation history** on the detail page: priority change, agent change, due-date change, reason, who, when.
5. **The triage result** after a successful create — the applied priority, the due date, the assigned agent, and the "unassigned, because..." case.
6. **The escalate control disabled or hidden** when the ticket cannot be escalated (already `Critical`, or `Resolved` / `Closed`) — *and* the server's rejection still handled, because UI state goes stale.

Everything you touch must be typed. **No** `any`, **no** `@ts-ignore`.

2.5 Optional stretch — the SLA summary report

Do this only if Tasks 1–3 are finished and tested.

`GET /api/reports/sla-summary?from=&to=&groupBy=agent|priority` — per group: total tickets, how many breached / at risk / within / met, the breach rate, and the average resolution hours. **It must be one database-side aggregation**, not tickets pulled into memory and grouped in C#.

2.6 Tests for Task 2

Backend (`apps/api/tests/SupportDesk.UnitTests`, xUnit + Moq — reuse `FixedClock` and `TicketBuilder`):

- SLA window for each priority, for both customer tiers, including the 1-hour floor
- Forced- `Critical` categories, and that a requested priority is respected otherwise
- Agent selection: fewest open wins · specialists only where required · agents at their limit excluded · inactive agents excluded · ties deterministic · **nobody eligible → unassigned with a reason, not an exception**

- Escalation: priority steps up one · due date recomputed from *now* · assignee kept when still eligible and changed when not · the history row holds the right before/after values · `Critical` and `Resolved / Closed` are rejected
- SLA status at the boundaries: exactly on the due date, one tick after it, exactly at the 25% threshold, resolved-before vs resolved-after

Frontend (Vitest + React Testing Library, tests live beside the code):

- A too-short reason blocks submission, shows a field error, and **does not call the API**
- A `409` from the escalate endpoint is shown to the user and the ticket is not changed on screen
- The SLA indicator renders each state distinguishably

Task 3 — Login and authentication

The problem we are solving

The support desk is open to anyone who can reach the URL. Every API endpoint answers to any caller, and the app has no idea who is using it. We want the support agents who already exist in the database to sign in, and everything else to be closed to anyone who has not.

3.1 What to build — backend

#	Requirement
AU-1	Agents can sign in with an email address and a password. The six seeded agents are the users. Add credentials to them (a <code>PasswordHash</code> column on <code>Agent</code> , or a separate <code>Users</code> table linked to an agent — your call, justify it in the README) with an EF Core migration you commit.
AU-2	Passwords are stored hashed and salted — <code>PasswordHasher<T></code> from ASP.NET Core Identity, BCrypt, or PBKDF2 via <code>Rfc2898DeriveBytes</code> . Never plain text, never reversible encryption, never your own hash function.
AU-3	Seed the development credentials , hashed at seed time, in the seeder — and document the development password in the seeder or <code>.env.example</code> . It is a local fixture, so it must be obviously not a real credential, and the hash is what goes in the database.
AU-4	<code>POST /api/auth/login</code> takes <code>{ "email": "...", "password": "..." }</code> and returns a JWT plus its expiry and enough about the user for the UI to greet them. Wrong email and wrong password must return the same <code>401</code> with the same generic message — do not tell an attacker which half was right. An inactive agent (Yuki) cannot log in.
AU-5	<code>GET /api/auth/me</code> returns the currently authenticated user, or <code>401</code> .
AU-6	Every existing endpoint requires a valid token. <code>/api/tickets</code> , <code>/api/customers</code> , <code>/api/agents</code> , <code>/api/categories</code> and everything you added in Task 2 must return <code>401</code> without one. Apply this centrally — a global policy or a fallback policy — not by remembering to decorate each action. <code>POST /api/auth/login</code> and Swagger stay open.
AU-7	The signing key comes from configuration (<code>appsettings.Development.json</code> or an environment variable) and is added to <code>.env.example</code> . It must not be a literal in a <code>.cs</code> file, and no real secret may be committed. Set a sensible token lifetime and say what you chose.

#	Requirement
AU-8	<code>escalatedBy</code> now comes from the token, not the request body (see Task 2B). Escalation history must record who actually did it.

3.2 What to build — frontend

#	Requirement
AU-9	A login page at <code>/login</code> : email and password fields, labelled and accessible, client-side validation, a pending state on submit, and a clear error message when the credentials are rejected.
AU-10	The token is stored and attached to every API call — extend the existing client in <code>apps/web/src/api/client.ts</code> rather than sprinkling headers through components. Say in your README where you chose to keep the token and what the trade-off is.
AU-11	Protected routes. An unauthenticated visitor to any page is sent to <code>/login</code> ; after a successful login they land on the page they were trying to reach (or the ticket list).
AU-12	The signed-in agent's name is visible , with a logout that clears the session and returns to <code>/login</code> .
AU-13	A 401 from any call (for example an expired token) clears the session and sends the user to <code>/login</code> instead of showing a broken screen.

3.3 Tests for Task 3

At least three that test behaviour — for example:

- Logging in with the wrong password returns `401` and issues no token
- A protected endpoint returns `401` without a token and `200` with one
- Password verification succeeds for the right password and fails for the wrong one
- The frontend redirects an unauthenticated user to `/login`, or attaches the token to requests

3.4 Explicitly **not** wanted in Task 3

Sign-up or self-registration · password reset or "forgot password" · email verification · OAuth / SSO / an external identity provider · refresh-token rotation · multi-factor auth · account lockout or rate limiting · a roles-and-permissions matrix · HTTPS certificate work.

If you think one of those matters, write a sentence about it under **Known limitations** rather than building it. (*A single role check — for example, only a lead may escalate — is a reasonable stretch if everything else is finished, but it is not required.*)

Task 4 — Dockerise the API

The problem we are solving

Today the API only runs from a developer's machine with the .NET SDK installed, and every new team member spends their first morning getting it to start. We want the API packaged as a container image, so that one

command brings up a working backend — the same way on every machine, and the same image that would later be deployed.

4.1 What to build

#	Requirement
DK-1	A multi-stage Dockerfile at <code>apps/api/Dockerfile</code> , built from the repository root as the build context: build and publish with the .NET 10 SDK image, run on the much smaller ASP.NET runtime image. Copy the project files and <code>Directory.Build.props</code> / <code>Directory.Packages.props</code> / <code>global.json</code> and restore before copying the rest of the source, so a code change does not throw away the cached restore layer.
DK-2	A .dockerignore that keeps <code>bin/</code> , <code>obj/</code> , <code>node_modules/</code> , <code>.git/</code> , <code>.env</code> and the web app out of the build context.
DK-3	The container runs as a non-root user and listens on HTTP port 8080 inside the container. TLS is terminated in front of the container in a real deployment, so no certificates go into the image.
DK-4	No configuration or secret is baked into the image. The connection string, the JWT signing key from Task 3 and the environment name all come from environment variables at run time. Anything new goes into <code>.env.example</code> .
DK-5	An api service in docker-compose.yml , next to the existing <code>sqlserver</code> one: built from your Dockerfile, published on <code>localhost:5080</code> , connecting to SQL Server by its service name (not <code>localhost</code>), and only starting once SQL Server is healthy (<code>depends_on</code> with <code>condition: service_healthy</code>).
DK-6	The database is ready when the API is. On <code>docker compose up</code> against an empty volume, the API applies its migrations and seeds the demo data, as it does today in Development. Say in your README how you would run migrations in a real deployment instead, and why.
DK-7	A health endpoint at <code>GET /health</code> that reports whether the API can reach its database. It stays open without a token — add it to the exceptions from AU-6, centrally.
DK-8	The web app can use the containerised API: <code>VITE_API_PROXY_TARGET=http://localhost:5080 npm run web</code> works, and your README says so.

4.2 How to verify Task 4

There is no automated test to write for this task. Instead, check — and describe in your README how you checked — that:

- `docker compose down -v && docker compose up --build` from a clean checkout ends with a healthy API at <http://localhost:5080/health> and seeded data at `/api/tickets` (with a token, once Task 3 is done)
- Changing one `.cs` file and rebuilding reuses the cached restore layer
- `docker image ls` shows a runtime image far smaller than the SDK image — say how big yours is
- `docker compose exec api whoami` does not print `root`

4.3 Explicitly **not** wanted in Task 4

Kubernetes or Helm · pushing to a registry · CI/CD pipelines · HTTPS inside the container · dockerising the React app · Docker secrets or a vault · multi-architecture builds.

If you think one of those matters, write a sentence about it under **Known limitations** rather than building it.

Where to focus your effort

This is what we score, and how to spend the two days well.

Area	Weight	What earns marks	What loses marks
OOP & SOLID	13%	Rules in small, separately testable units; create and escalate share them; interfaces where there is a real seam	Business logic in a controller or a React component; one 300-line service; an interface per class for its own sake
.NET / backend	14%	Correct status codes (<code>409</code> for escalation conflicts, <code>401</code> for missing auth), validation on every request, async all the way down, the central error handling reused	<code>.Result</code> , <code>catch { }</code> , EF used straight from a controller, a <code>500</code> where a <code>409</code> or <code>401</code> belongs
React / TypeScript	13%	One reusable SLA indicator, a custom hook for fetching, loading/error/empty states, full typing, labelled and <code>aria</code> -wired inputs	<code>any</code> , <code>fetch</code> inside components, missing error handling, duplicated markup, prop drilling
Security & authentication	10%	Hashed and salted passwords, generic auth failure messages, the key in configuration, authorization applied centrally, the actor taken from the token, nothing secret committed or baked into the Docker image, a non-root container	Plain-text or home-made hashing, a hard-coded signing key, <code>[Authorize]</code> forgotten on half the endpoints, "user not found" vs "wrong password", a token logged or put in a URL, a secret in the <code>Dockerfile</code>
Debugging	10%	The defect fixed at the cause, with a regression test, explained in your own words	Fixing the symptom: filtering in memory, <code>reload()</code> , a remount <code>key</code> , an effect with no dependency array
Testing & code quality	10%	Boundary cases, a fake clock, readable names, no duplicated rule logic, no magic numbers	Trivial tests; over-mocking; dead code and commented-out blocks
SQL / database	9%	A proper migration, <code>AsNoTracking()</code> on reads, narrow projections, no N+1, the list filters and the SLA filter translated to SQL before paging, a deliberate index	<code>.ToList()</code> then filtering in C#, counting in memory, loading whole object graphs, no migration
Design patterns	8%	Two or three that genuinely fit (a strategy/policy for the SLA, a factory for triaged creation, a shared query object), each with a <i>why</i> in the README	Naming patterns you cannot justify; MediatR/AutoMapper/Redux added for a handful of endpoints
Git & documentation	5%	Small focused commits, the defect fix and the filtering committed separately from features, a PR a reviewer can read in two minutes	One giant commit; <code>bin/</code> , <code>node_modules/</code> or a secret committed

Area	Weight	What earns marks	What loses marks
Requirements & delivery	4%	Edge cases handled (nobody eligible, already Critical, inactive agent); assumptions written down	Silently skipping parts of the spec
Effective AI use	4%	An honest account: what you used, what you rejected and why, how you verified it	"Used ChatGPT for everything"; generated code you cannot explain

The four things that most often separate submissions:

1. **Where the rules live.** If create and escalate each compute the SLA their own way, you lose marks in four categories at once. Write the rule once.
2. **Whether the database does the work.** We read the SQL your code generates.
3. **Whether security is structural.** One central policy beats twelve `[Authorize]` attributes, eleven of which are right.
4. **Whether you can explain it.** We will pick a file and ask you to walk through it. Use AI freely — just do not ship anything you would not want to explain out loud.

Suggested two-day plan

A guide, not a rule.

Day 1 — understand, fix, and build the core

Time	Focus
0:00–1:00	Step 0. Run everything, use the app, read the code in the order listed.
1:00–2:00	Task 1.1 Reproduce and fix DEFECT-117, with a regression test. Commit it on its own.
2:00–3:30	Task 1.2 Server-side filtering, its tests, and a look at the SQL it generates. Commit it on its own.
3:30–4:00	Design before you type. Where does each rule live? What is the one abstraction worth having? What will you deliberately <i>not</i> abstract? Write it into your README now, while it is fresh.
4:00–6:15	Task 2 domain + application: SLA calculation, the priority rule, agent selection, the escalate use case — shared by create and escalate.
6:15–7:30	Unit tests for those rules and their edge cases.

Day 2 — persist, expose, secure, finish

Time	Focus
0:00–1:15	Escalation entity + EF configuration + migration + index; the history query.

Time	Focus
1:15–2:15	Endpoints, DTOs, validation, status codes, the extended create response, the <code>slaStatus</code> filter — then read the generated SQL and check it does what you think.
2:15–4:15	Task 2 frontend: types, API client, hook, SLA indicator, filter, escalation panel, history, triage result.
4:15–6:15	Task 3 : credentials + migration + seed, <code>login / me</code> , the global authorization policy, then the login page, token handling, protected routes and logout.
6:15–7:00	Tests for Task 3; re-run everything end to end while logged in.
7:00–8:15	Task 4 : Dockerfile, <code>.dockerignore</code> , the <code>api</code> compose service, <code>/health</code> ; then bring the whole stack up from an empty volume.
8:15–9:00	README, tidy the commits, open the PR, and read your own diff before you submit.

Definition of done

Task 1 — defect and filtering

- ☐ Changing any single filter sends a new request immediately, with no request per keystroke
- ☐ A regression test for DEFECT-117 that fails on the original code
- ☐ Filtering by `High` returns only High tickets, and the count matches
- ☐ Status, priority, category, customer, agent, unassigned-only and search all filter, alone and combined
- ☐ Filters are applied in SQL before counting and paging; the pager is right
- ☐ Tests for the filtering path: exact-match priority, combined filters, search on customer name

Task 2 — escalation & assignment

- ☐ A `Security` or `Outage` ticket comes out `Critical` however it was requested
- ☐ A Premium customer gets half the window, never below 1 hour
- ☐ Specialist categories only ever auto-assign to a specialist
- ☐ The eligible agent with the fewest open tickets wins, deterministically; full and inactive agents are skipped
- ☐ Nobody eligible → `201`, unassigned, with a reason — not an error
- ☐ Escalation raises priority one level, recomputes the due date from now, re-evaluates the owner, and writes an immutable history row
- ☐ Escalating a `Critical` / `Resolved` / `Closed` ticket → `409`; unknown ticket → `404`; bad reason → `400`
- ☐ `GET /api/tickets/{id}/escalations` returns the history with before/after values
- ☐ `?slaStatus=Breached` works, in SQL, with correct paging and count
- ☐ A committed migration that applies cleanly to an empty database
- ☐ SLA indicator in the list **and** on the detail page, from one component
- ☐ Escalation panel: validation, pending state, server errors, refresh on success
- ☐ Escalation history and the triage result are visible in the UI

- ☐ The escalate control is unavailable when the ticket cannot be escalated

Task 3 — authentication

- ☐ A seeded agent can log in with email + password; the inactive agent cannot
- ☐ Passwords are stored hashed and salted; no plain text anywhere, including the seeder
- ☐ Wrong email and wrong password return the same generic 401
- ☐ Every non-auth API endpoint returns 401 without a valid token, applied centrally
- ☐ The signing key comes from configuration and is in .env.example; no secret committed
- ☐ The UI has a login page, a visible signed-in user, and a logout
- ☐ Unauthenticated visitors are redirected to /login; a 401 mid-session sends them back there
- ☐ escalatedBy comes from the authenticated user

Task 4 — Docker

- ☐ A multi-stage apps/api/Dockerfile with a cached restore layer, running on the ASP.NET runtime image
- ☐ A .dockerignore that keeps build output, node_modules/, .git/ and .env out of the context
- ☐ The container runs as a non-root user on port 8080
- ☐ No connection string, signing key or other secret in the image; new variables in .env.example
- ☐ docker compose up --build on an empty volume gives a migrated, seeded API on localhost:5080
- ☐ GET /health reports database connectivity and is reachable without a token
- ☐ The web app works against the containerised API

Overall quality

- ☐ npm run build and npm test (both suites) pass from the repository root, with no new warnings
- ☐ npm run lint is clean; no any, no @ts-ignore
- ☐ No business rule inside a controller or a React component
- ☐ README updated (see below)
- ☐ Readable commit history; no build output, node_modules/ or secrets committed

What to write in your README

Update README.md, or add a section to it, covering:

- **What you implemented** — and the business rules as you understood them
- **Design decisions** — what you chose, what you rejected, and why. This is the section we read most carefully. Two or three honest paragraphs beat two pages.
- **The defect** — root cause, fix, how to verify
- **Filtering** — where the filters are applied, how you checked the SQL, and how the next filter gets added
- **Authentication** — where credentials live, how passwords are hashed, how the token is issued and stored, the token lifetime, and what you would do differently in production
- **How to log in** — the development credentials, so we can run your submission

- **Docker** — how to build and run the API container, the environment variables it needs, the image size, and how you would run migrations in a real deployment
- **Database** — what you added, and which index you added and why
- **Testing** — how to run the tests, what you chose to test, what you deliberately did not
- **AI usage** — see below
- **Known limitations** — anything incomplete, simplified, or that you would do differently. This section helps you; it never counts against you.

Keep it practical: one to three pages.

Using AI

Use whatever you normally use. We are not testing whether you can work without it — we are testing whether you stay in control of the result.

In your README, tell us:

- Which tools you used, and for what (code, tests, debugging, SQL, docs, learning)
- A prompt or workflow that worked well
- Something AI generated that you **accepted** largely as-is
- Something AI generated that you **changed or rejected**, and why
- Any bug AI introduced, or helped you find
- How you **verified** generated code before committing it — tests? the debugger? the generated SQL? reading the documentation?

An honest "Copilot suggested storing the password with SHA-256 and no salt, which I replaced with `PasswordHasher<T>` " is worth far more to us than a polished paragraph that says nothing.

How to submit

Preferred:

```
git checkout -b feature/ticket-escalation
# ... work, committing as you go ...
git push -u origin feature/ticket-escalation
# then open a Pull Request
```

Alternative: push to your own GitHub repository — with your real commit history, not one squashed "initial commit" — and send us the link (add us as a collaborator if it is private).

Your PR description should tell a reviewer in two minutes: what changed, the root cause of the defect, your main design decisions, and anything you left out. There is a template in `.github/pull_request_template.md`.

Send us the link, the development login credentials, and a rough note of how long you spent and on what.

Afterwards we will have a short conversation about your design decisions, your tests and your AI usage. Expect "why did you do it this way?", "what would you change if this had 10 million tickets?", "what is the weakest part

of your authentication?" and "what would you change in your image before running it in production?" — thinking about those now is time well spent.

Out of scope

Do **not** build: user registration or password reset, roles and permission matrices, OAuth/SSO, refresh tokens, multi-factor auth, notifications, background jobs, real-time updates, microservices, deployment beyond the Docker image in Task 4, CI, caching layers, event sourcing, multi-tenancy, internationalisation, or a visual redesign.

If you think one of those would help, write a sentence about it under Known limitations instead of building it.

If something is ambiguous

Make a reasonable decision, write it down in your README, and keep going. That is exactly what we would want you to do on the job, and far better than a blocked afternoon.

Good luck.